JAWVEE

Spatial Teaching Engine

Deep-Research Product Requirements Document

Purpose: Define a scalable architecture that enables an AI tutor to understand arbitrary student questions, reason over a persistent canvas, render accurate mathematical/scientific representations, and synchronize speech, drawing, pointing, highlighting and interaction as one coherent teaching performance.

Core requirement: No hardcoded question-specific behaviors, coordinate patches, special-case scripts, or brittle visual heuristics. The system must generalize through typed representations, compositional reasoning, capability discovery, deterministic execution, uncertainty management, and closed-loop verification.

Research basis: Multimodal spatial reasoning, referring-expression grounding, dynamic scene graphs, handwriting mathematical-expression recognition, and realtime voice-agent architecture.

1. Executive Summary

JAWVEE should not be implemented as an LLM that receives a screenshot of a whiteboard and then attempts to describe or manipulate it. That architecture places too much responsibility on probabilistic visual inference. Current multimodal research continues to show substantial weaknesses in complex spatial reasoning, visual grounding, ambiguous references, multi-turn consistency, and high-level planning. ■cite■turn0academia24■turn0search1■turn0search4■

The recommended architecture is a **spatially grounded teaching operating system**. The canvas is a structured environment; the visual rendering is one projection of that environment. The AI operates over a canonical scene graph, semantic knowledge graph, lesson state, student state, event history and temporal teaching timeline. Vision is used only where pixels contain information unavailable from structured state.

The second critical architectural change is to treat teaching as a **performance**, not as a chatbot response. Speech, drawing, pointing, highlighting, camera movement and waits are represented as one synchronized teaching plan. An execution runtime schedules these actions against an audio/media clock and can interrupt, invalidate, verify and repair the plan when the student changes direction.

Absolute 100% accuracy for arbitrary natural-language interpretation is not technically honest. The product can, however, make large portions of the system deterministic and guarantee correctness within explicit domains. The remaining probabilistic steps must be confidence-gated: when JAWVEE cannot identify the student's referent or intended operation above a threshold, it must ask or confirm rather than guess.

The resulting design scales to questions such as “why is the sky blue?”, algebra, calculus, geometry, physics, biology, chemistry and future domains because the system is not built around question templates. It decomposes each question into **intent** → **concepts** → **claims** → **evidence/visual primitives** → **explanation plan** → **synchronized actions**.

2. Product Definition

Feature: JAWVEE Spatial Teaching Engine — the runtime responsible for turning an arbitrary student question or lesson objective into a grounded explanation that can speak, draw, point, highlight, animate and adapt in real time.

Dimension	Definition
Primary user	Student interacting conversationally with JAWVEE
AI role	Tutor that explains concepts using voice + spatial visual demonstration
Canvas role	Persistent structured teaching environment, not merely a drawing surface
Source of truth	Versioned semantic scene graph + world state + event log
Visual input	Canvas-native objects/strokes plus selective perception of pixels
Output modalities	Speech, text, drawing, pointer, highlight, camera, animation, question/wait
Scalability principle	Capability-driven, schema-driven, compositional; no question-specific patches
Reliability principle	Deterministic where possible; confidence-gated where inference is unavoidable

3. First-Principles Requirements

The system should be designed from the following invariants. These are architectural laws, not implementation preferences.

- **Invariant 1 — One world:** speech, canvas, pointer and reasoning must reference the same object/relationship identifiers.
- **Invariant 2 — One truth:** the scene graph/world model is authoritative; screenshots are observations, not the source of truth.
- **Invariant 3 — No hidden state:** every consequential canvas mutation creates a typed event and version.
- **Invariant 4 — No arbitrary coordinates from the LLM:** the model requests semantic targets; the renderer resolves geometry.
- **Invariant 5 — No unsupported certainty:** uncertain perception or reference resolution cannot silently become fact.
- **Invariant 6 — No LLM arithmetic when deterministic computation is available:** symbolic/numeric engines validate mathematical claims.
- **Invariant 7 — Speech and visuals are one plan:** visual actions are anchored to semantic speech cues or explicit timeline cues.
- **Invariant 8 — Interruptions invalidate stale plans:** a student question can cancel future actions from an old teaching plan.

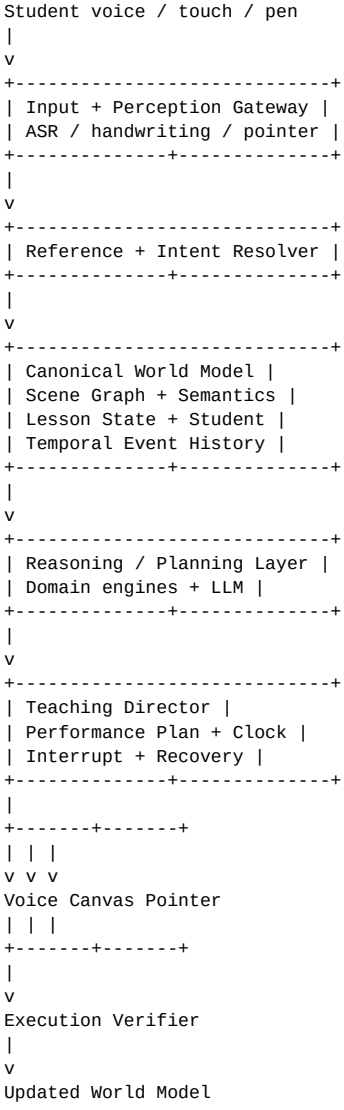
- **Invariant 9 — Every important action is verifiable:** execute → observe → validate → commit or repair.
- **Invariant 10 — Generalization through composition:** new questions should be solved by combining existing primitives, not by adding special cases.

4. Why Screenshot-First Architecture Fails

A screenshot contains pixels but does not inherently contain stable identity, semantics, causal history, mathematical structure, object provenance, or the relationship between an object and the words spoken about it. Research on visual grounding and multimodal dialogue shows persistent difficulty with ambiguous, multi-turn and relational references; current spatial benchmarks also report gaps between perception and higher-level symbolic reasoning/planning.
■cite■turn0search0■turn0search1■turn0academia24■

Question	Screenshot-first answer	Required JAWVEE answer
What is this?	Infer from pixels	Resolve semantic object ID
Which angle?	Guess from image/context	Reference resolver + pointer + recency + geometry
What changed?	Compare images	Event log/version diff
Where should I point?	Predict pixels	Target object → anchor geometry
Is equation correct?	Visual/LLM judgment	Symbolic parser + deterministic validator
What did student erase?	Maybe infer	Event history
What should I draw?	Generate pixels	Plan semantic objects + renderer

5. Canonical System Architecture



The LLM is not the runtime. It is one reasoning component inside a larger stateful system. The Teaching Director is responsible for execution semantics, temporal coordination and cancellation. The renderer is responsible for exact geometry. Domain engines are responsible for deterministic calculations and representations.

6. Canonical Scene Graph

6.1 Object identity

Every meaningful visual entity receives a stable ID that survives rendering, highlighting, movement and conversation. IDs are generated by the scene manager, never by the model as arbitrary strings that can collide.

```
object_id: "scene_8f2.../angle/instance_17"
type: "angle"
semantic_type: "right_angle"
geometry:
anchors: [ray_x, vertex_o, ray_y]
region: ...
relationships:
- vertex_of: ...
- formed_by: [...]
provenance:
creator: "ai"
source_event: "evt_..."
state:
visibility: "visible"
confidence: 1.0
```

6.2 Geometry + semantics

Separate **what an object is** from **where it is rendered**. A semantic object may have multiple visual projections. For example, an angle can be represented by two rays, a vertex, a curved marker and a 90° label. The semantic object remains stable if the visual representation changes.

6.3 Relationships

Use typed edges: contains, part_of, intersects, perpendicular_to, parallel_to, labels, represents, derived_from, points_to, measured_by, caused_by, adjacent_to, inside, outside, above, below, before, after, and domain-specific relations. A graph representation is preferable to flat object lists because many educational questions are relational.

Research on scene-graph conditioning and dynamic scene-graph updating supports the broader principle that structured relational representations can improve grounded reasoning and that dynamic environments require continuous state updates rather than static interpretation. ■cite■turn2academia0■turn2academia2■

7. Domain-Neutral Semantic Layer

Do not create one giant hardcoded taxonomy. Use extensible typed schemas with a domain registry. Each domain supplies concepts, relations, invariants, visual primitives and validators.

Layer	Example
Entity	triangle, DNA molecule, cloud droplet
Property	area, mass, wavelength, charge
Relation	contains, perpendicular, bonds_to
Operation	differentiate, balance, rotate, compare
Constraint	sum of triangle angles = 180°
Evidence	observation, law, theorem, retrieved source
Visual primitive	axis, vector, bond, membrane, ray
Validator	symbolic solver, geometry engine, domain checker

This allows a new domain to be added by registering domain knowledge and renderers rather than changing the core conversation engine.

8. Arbitrary Question → Teaching Plan

The core generalization mechanism is a compositional compiler-like pipeline.

```
Student question
↓
Intent interpretation
↓
Concept/entity extraction
↓
Reference resolution
↓
Question decomposition
↓
Knowledge/evidence retrieval
↓
Reasoning + domain validation
↓
Pedagogical strategy
↓
Visual explanation plan
↓
Semantic scene operations
↓
Teaching Performance Plan
↓
Speech + canvas + pointer + timing
↓
Verification / repair
```

The model does not need a predefined answer template for “why is the sky blue?”. It needs to identify that the student is asking for a causal explanation, construct the relevant conceptual chain, choose an appropriate visual abstraction, and compile that into available visual primitives.

8.1 Example: Why is the sky blue?

```
Question: "Why is the sky blue?"

Intent:
causal_explanation

Concept graph:
sunlight
-> broadband visible spectrum
-> atmosphere / molecules
-> wavelength-dependent scattering
-> shorter wavelengths scatter more strongly
-> scattered light enters observer's field of view
-> sky appears blue

Pedagogical level:
age/knowledge model determines abstraction

Visual plan:
1. draw Sun
2. draw atmosphere boundary
3. emit rays
4. show long/short wavelength representations
5. animate scattering
6. highlight scattered short-wavelength paths
7. point to observer-facing sky region

Speech anchors:
"sunlight"
"atmosphere"
"shorter wavelengths"
"scatter"
"sky"

Validation:
domain/evidence checker verifies causal claims
visual verifier checks objects/actions
```

The important point is that the system does not contain a “sky-blue script.” It contains reusable concepts such as source, medium, ray, wavelength, interaction, scattering, observer, highlight and causal relation. The planner composes them for this question.

9. Question Types the Engine Must Generalize Across

Question family	Example	Required reasoning mode
Definition	What is mitosis?	Concept → components → concise visual

Question family	Example	Required reasoning mode
Causal	Why is the sky blue?	Cause graph → mechanism animation
Procedural	How do I solve 2x+5=15?	State transition → verified steps
Comparative	Why is a mammal different from a reptile?	Feature graph → side-by-side visual
Counterfactual	What if Earth had no atmosphere?	World-model perturbation
Spatial	Why is this angle 90°?	Reference resolution → geometry proof
Diagnostic	Where did I go wrong?	Event history → error localization
Predictive	What happens if force doubles?	Domain model → simulation/derivation
Socratic	What should we do next?	Pedagogical policy → guided prompt
Meta	Why did you draw this?	Teaching-plan provenance → explain intent
Ambiguous	Why does this happen?	Resolve references; ask if confidence low
Multi-hop	Why does A cause C if B is in between?	Graph traversal + causal reasoning
Visual request	Can you show me?	Construct visual plan from concepts

10. Visual Primitive System

The renderer should expose a domain-neutral set of semantic primitives rather than raw SVG/canvas commands to the LLM.

- Create/modify semantic object.
- Draw path/stroke.
- Create line/ray/vector/axis.
- Create geometric shape.
- Create mathematical expression.
- Create graph/chart.
- Create diagram node + relation.
- Create label tied to an object.
- Highlight/focus/pulse an object or region.
- Circle/underline/bracket an object.
- Point/trace an object.
- Animate an existing relationship or transformation.
- Pan/zoom camera to a semantic region.
- Erase/hide/revert an object.
- Group/ungroup objects.

The LLM should say *highlight(angle_17)*, not *draw a yellow ellipse from x=381 to x=426*. The geometry service resolves the correct bounds/anchor and the renderer performs the exact operation.

11. Constraint-Based Rendering

For mathematical and scientific content, rendering should be generated from semantic constraints rather than freehand pixel generation.

Example: to draw perpendicular axes, specify **two infinite/finite lines, common origin, 90° relation, labels, viewport fit**. The geometry engine computes endpoints. To draw a triangle with a right angle, specify vertices and the right-angle constraint. The renderer derives the marker.

This is fundamentally more scalable than hardcoding visual coordinates for every lesson. It also allows resizing, zooming, localization and accessibility without changing the semantic model.

11.1 Renderer contract

```
Input:
semantic object graph
constraints
style tokens
viewport
animation spec

Output:
deterministic render tree
stable object-to-pixel mapping
hit-test geometry
accessibility metadata
render version
```

12. Handwriting and Student Drawing

Freehand student input is one of the places where pixels genuinely contain new information. The system should retain raw strokes and also create hypotheses about their semantic interpretation.

Modern work on structural handwritten mathematical expression recognition is particularly relevant: recognition that explicitly links predicted symbols to raw traces and spatial relations is more useful for interactive educational systems than a black-box LaTeX string because it supports selective updates and error analysis. ■cite■turn2academia3■

```
Raw:
stroke_1, stroke_2, ...

Perception hypothesis:
fraction_3
numerator = "2"
denominator = "3"
trace_ids = [...]

Confidence:
0.94

Status:
probable

Promotion rule:
confirmed only after threshold / contextual corroboration
```

Never destroy the raw evidence. A later recognizer, improved model or student correction must be able to reinterpret the original strokes.

13. Reference Resolution Engine

This is the central mechanism for handling millions of different ways a child can ask the same question.

Expressions such as “this”, “that”, “these two”, “the one on the left”, “the blue thing”, “why does this go here?”, “what happens if I move this?”, or “why is this 90 degrees?” must be resolved against a dynamic candidate set.

13.1 Candidate generation

- Objects currently visible.
- Objects under/near the student's pointer.
- Objects recently created or referenced.
- Objects in the current teaching focus.
- Objects semantically compatible with the noun phrase.
- Objects satisfying spatial descriptions.
- Objects participating in the current concept/relation.
- Objects mentioned in recent speech.

13.2 Candidate scoring

```
score(object) =
language_match
+ pointer_proximity
```

```
+ gaze/attention signal (if available)
+ dialogue_recency
+ current_focus
+ spatial_relation_match
+ semantic_relation_match
+ visual_attribute_match
+ temporal_creation_match
- contradiction_penalties
```

The exact scoring implementation should remain replaceable. The contract is that the resolver returns candidates, evidence and confidence rather than silently asserting a guess.

13.3 Ambiguity policy

Confidence	System behavior
>= high threshold	Act directly
middle range	Use contextual confirmation or conservative action
low	Ask a clarification question
no viable candidate	State that the reference is unclear

14. Multimodal Reference Resolution

Language alone is insufficient for reliable grounding. Research specifically identifies ambiguity, complex multi-relation expressions and negation as difficult for visual-language models, and dynamic multimodal dialogue benchmarks combine speech, motion, gaze and scene geometry for this reason. ■cite■turn0search1■turn0search2■

JAWVEE should therefore fuse at least four signals: linguistic reference, canvas geometry, interaction state and temporal dialogue context.

```
"Why is THAT 90 degrees?"

Speech referent candidates:
angle_17, angle_22

Student pointer:
within 14 px of angle_17

Current lesson focus:
angle_17

Last teacher phrase:
"This angle is 90 degrees."

Resolution:
angle_17
confidence = high
evidence = [pointer, recency, focus, language]
```

15. World Model

The world model is the authoritative state of the lesson at a particular version. It should be queryable without asking the LLM to reconstruct the board from conversation history.

```
WorldState
session
lesson
viewport
scene_graph
student_state
teaching_state
audio_state
interaction_state
pending_plan
event_cursor
knowledge_context
uncertainty_state
```

15.1 Temporal state

Every state mutation produces a version. The system can therefore answer: What existed when the student asked the question? What had the teacher just said? Which objects were visible? Which actions were already completed? Which future actions were

queued but not yet executed?

16. Teaching Performance Protocol

Speech and visual actions must be represented together in a typed performance plan.

```
Performance
id
base_world_version
intent
speech_segments[]
visual_actions[]
pointer_actions[]
camera_actions[]
waits[]
interruption_policy
verification_requirements

SpeechSegment
id
text
semantic_anchors[]
audio_reference

VisualAction
id
operation
target_object_ids[]
trigger
duration
verification_rule
```

A speech segment can contain anchors such as “X-axis”, “this angle”, “shorter wavelengths” or “the numerator”. The runtime binds those anchors to object IDs before or during playback.

This avoids a fragile timing-only approach. Timing remains important, but semantic anchors are the primary synchronization mechanism; audio timing is the temporal clock.

Browser speech APIs expose word/sentence boundary events, but browser support is not uniform, so production synchronization should rely on a controlled audio pipeline where possible rather than assuming identical browser timing.
■cite■turn1search0■turn1search7■

17. Audio-Clock Synchronization

The media playback clock should be the authoritative clock for speech-linked visual cues. Do not schedule visuals from LLM generation time or fixed delays.

```
LLM plan
↓
TTS / realtime voice generation
↓
audio timeline
↓
cue scheduler
↓
canvas / pointer / highlight

If speech speed changes:
semantic cue remains attached to the relevant speech position

If user interrupts:
current playback position becomes known
unplayed plan is invalidated
world state is preserved
```

Current realtime voice architectures increasingly separate low-latency media flow from deeper reasoning and tool use; this separation is directly applicable to JAWVEE. OpenAI's current realtime architecture describes a full-duplex media path, asynchronous delegation for deeper reasoning/tool use, and explicit handling of interruption and state.
■cite■turn1search1■turn1search4■

18. Teaching Director

The Teaching Director is the runtime orchestrator. It is not a second conversational brain. Its job is to convert an approved teaching plan into safe, synchronized, interruptible execution.

- Validate plan against current world version.
- Resolve semantic targets to current geometry.
- Schedule actions against audio/media clock.
- Execute canvas mutations.
- Coordinate pointer and camera.
- Track completed/active/pending actions.
- Detect student interruption.
- Cancel stale future actions.
- Request verification.
- Repair failed actions.
- Commit completed teaching step.
- Emit telemetry for evaluation.

19. Transactional Execution

Treat consequential teaching actions as transactions.

```
PLAN
↓
VALIDATE
↓
EXECUTE
↓
OBSERVE
↓
VERIFY
↓
COMMIT

Failure:
↓
REPAIR / ROLLBACK / ASK
```

For example, if JAWVEE intends to point at an angle, the action should not be considered complete merely because the tool call succeeded. The verifier must confirm that the pointer's semantic target and geometry correspond to the intended angle in the current scene version.

This is analogous to closed-loop control: the system does not assume that issuing an action means the desired state has been achieved.

20. Interruption and Barge-In

A child can interrupt at any moment. This is not an edge case; it is a core tutoring behavior.

```
Teacher plan:
draw triangle
explain side AB
highlight angle C
derive theorem

Student interrupts after side AB:
"Wait, why is that side equal?"

Required:
stop/cancel speech as appropriate
stop or freeze nonessential visual queue
preserve committed scene state
resolve "that side"
answer new question
invalidate stale downstream actions
re-plan from new state
```

Realtime voice systems explicitly address interruption and full-duplex interaction because rigid turn-based architectures produce brittle handoffs. JAWVEE should adopt the same principle while keeping whiteboard state under its own deterministic runtime. ■cite■turn1search1■turn1search4■

21. Student Question During Animation

If the student asks while a diagram is being drawn, the system needs a precise definition of what has already been perceived by the student.

Maintain two states: **committed world state** and **in-flight presentation state**. The committed state contains completed semantic actions. The presentation state contains animation progress and audio progress.

This allows JAWVEE to distinguish “the circle exists in the scene” from “the circle has only been 40% drawn.” The response planner can choose whether to finish, freeze, or cancel the animation depending on relevance.

Student interruption	Default policy
Question about current object	Freeze presentation; answer about current object
Question unrelated to current object	Preserve/finish only safe atomic action; switch focus
Correction of previous step	Rollback/reframe dependent future plan
Ambiguous reference	Do not continue risky visual action; clarify
Request to go faster/slower	Adjust future timing; preserve semantic plan

22. Knowledge and Evidence Layer

Open-ended questions cannot be solved purely by canvas mechanics. JAWVEE needs a knowledge/evidence layer that can retrieve authoritative information when the answer depends on facts outside the lesson model.

The knowledge layer should return claims with provenance, confidence and applicability. The teaching planner then chooses which claims to expose visually.

```
Claim
subject
predicate
object
domain
source
confidence
applicability
prerequisites
```

For science questions such as “why is the sky blue?”, the system should distinguish a scientifically supported causal explanation from a pedagogical simplification. The visual plan can be simplified while the underlying claim remains explicit and traceable.

23. Reasoning Architecture

Use a mixture-of-capabilities approach rather than asking one model to do everything.

Component	Responsibility
Realtime voice model	Listening, natural speech, immediate interaction
Reasoning model	Deep reasoning, decomposition, planning
Domain solver	Math/science symbolic computation and validation
Knowledge retrieval	External facts and evidence
Reference resolver	Language-to-scene grounding
Scene graph	Persistent spatial/semantic state
Renderer	Exact visual realization
Teaching Director	Temporal orchestration
Verifier	State/result correctness

A current OpenAI realtime architecture explicitly supports a fast voice model delegating deeper reasoning and tool use to backend models, which is conceptually aligned with this separation. ■cite■turn1search1■turn1search4■

24. Capability Registry — Eliminating Hardcoding

The system must discover what it can do through a typed capability registry. The LLM should not contain hidden assumptions about what a particular renderer supports.

```
Capability
id
domain
input_schema
output_schema
preconditions
invariants
cost
latency
visual_effect
verification_method

Examples:
geometry.construct_perpendicular
math.render_expression
biology.render_molecule
generic.highlight
generic.pointer_target
generic.camera_focus
```

When a new renderer or domain capability is added, the planner can discover it through the registry. This prevents the core model prompt from becoming a growing list of hardcoded cases.

25. Composition Rather Than Patching

Suppose a new question is “What would happen to the sky if Earth had no atmosphere?” The system should not need a special case. It composes existing capabilities: planet, atmosphere, sunlight, scattering, observer, counterfactual state, compare, animate, explain.

Similarly, “Why does DNA have two strands?” composes molecule, strand, base pair, complementary relation, bond, structure, highlight, compare and causal explanation.

The scalability test is therefore: **Can a new question be solved by adding or retrieving concepts and composing existing operations?** If the answer requires a new if-statement keyed to the wording of the question, the architecture is drifting toward patchwork.

26. Counterfactual and What-If Questions

Many student questions modify the current world: “What if I move this point?”, “What if gravity doubled?”, “What if there was no atmosphere?” These should be handled by branching the world model rather than mutating the canonical lesson state immediately.

```
Current World
|
+---- Branch A: canonical lesson
|
+---- Branch B: hypothetical change
|
+-- recompute domain constraints
+-- render differences
+-- explain causal consequences
+-- discard or commit based on user request
```

This makes hypothetical teaching safe and reversible.

27. Mathematical Accuracy

Mathematics should be represented as structured expressions, constraints and transformations. The LLM may propose a derivation; a symbolic engine validates each transformation before it becomes authoritative.

```
Student:
2x + 5 = 15

Proposed step:
2x = 10

Validator:
subtract 5 from both sides → valid

Proposed step:
x = 5
```

```
Validator:
divide both sides by 2 → valid

World model:
verified derivation chain
```

This enables exact highlighting: the AI can point at the term being transformed because the expression tree identifies it structurally, not because a vision model guessed its location.

For handwritten expressions, trace-to-symbol links should be retained where possible so the system can say “you changed this term” and map the explanation to the exact student strokes. ■cite■turn2academia3■

28. Science and Biology Accuracy

Scientific diagrams should use semantic graph representations where possible. A DNA drawing is not just a set of curves; it is a graph of entities and relations.

```
DNA molecule
■■ strand_1
■ ■■ nucleotide_1
■ ■■ nucleotide_2
■■ strand_2
■■ base_pair_1
■ ■■ adenine_1
■ ■■ thymine_1
■■ hydrogen_bond_group_1
```

The visual representation may be simplified for a child, but the underlying semantic graph should preserve the relationships required for later questions. This is how JAWVEE can answer follow-up questions without reinterpreting its own drawing from pixels.

For advanced biology, chemistry or physics, the domain layer should supply ontology terms, constraints and renderable primitives while the core spatial engine remains unchanged.

29. Sky-Blue Example — Full System Trace

```
Student:
"Why is the sky blue?"

1. Voice perception:
intent = causal explanation

2. Context:
current topic = light / atmosphere

3. Knowledge:
retrieve wavelength-dependent scattering explanation

4. Reasoning:
build causal graph

5. Pedagogy:
choose age-appropriate abstraction

6. Visual planner:
create sun, atmosphere, rays, observer
encode wavelength/scattering relationships

7. Performance planner:
speech segments + visual anchors

8. Teaching Director:
execute against audio clock

9. Student:
"Why do the blue ones scatter more?"

10. Reference resolver:
"blue ones" -> short-wavelength representation

11. Domain reasoning:
answer relative scattering behavior

12. Visual action:
highlight those rays + show scattering events
```

```
13. Verify:
highlighted target matches semantic object

14. Commit:
lesson state advances
```

At no point does the system need a hardcoded “sky is blue” canvas script. The question is decomposed into concepts and operations available in the domain registry.

30. Visual Verification

Verification should operate at three levels.

Level	Question
Semantic	Did the intended object/action exist and change state correctly?
Geometric	Is the rendered target actually at the intended location/relationship?
Perceptual	Does the resulting visual output look correct enough to a student/human observer?

Semantic verification is strongest because it is deterministic. Geometric verification uses the renderer's own scene data. Perceptual verification is a fallback for cases where rendering or external visual inputs can diverge from expected semantics.

Do not make a screenshot-based vision check the primary verifier when the renderer already knows the exact scene state. Use screenshots as an independent observation channel and for regression testing.

31. Failure Recovery

Failure	Detection	Recovery
Object missing	semantic verifier	recreate from plan
Wrong target	reference verifier	cancel + re-resolve
Pointer misaligned	geometry verifier	recompute anchor
Speech interrupted	audio state	truncate + replan
Student changes topic	intent resolver	invalidate future plan
Ambiguous question	confidence	clarify
Math inconsistency	symbolic validator	reject step
Rendering error	render version mismatch	rerender from canonical state
Network delay	runtime heartbeat	preserve world; resume/replan
Stale object ID	version conflict	resolve by semantic identity/version
Perception disagreement	multi-hypothesis state	request confirmation

32. Confidence and Abstention

The system must treat abstention as a successful outcome when evidence is insufficient. A tutor that says “I’m not sure which angle you mean—do you mean this one?” is more trustworthy than a tutor that confidently points to the wrong angle.

Confidence should be decomposed rather than a single opaque number where practical: linguistic confidence, spatial confidence, semantic confidence, temporal confidence and domain confidence.

```
ResolutionResult
candidates:
- object_id
evidence[]
confidence

selected: object_id | null
action_policy:
execute | confirm | clarify | abstain
```

33. Student Attention Model

To decide what “this” means, JAWVEE should maintain an attention model. Inputs may include pointer location, selection, dwell time, recent edits, speech references, current teaching focus, camera viewport and—if eventually available—gaze or other attention signals.

The attention model should not claim that the student looked at an object unless the system actually has a measurement. Treat inferred attention as evidence, not fact.

This mirrors the broader research direction in dynamic multimodal grounding, where synchronized speech, motion and scene geometry improve the representation of referring expressions. ■cite■turn0search2■

34. Temporal Memory

The system needs a temporal event log, not merely the latest canvas state.

```
t=10.2 AI creates angle_17
t=11.0 AI says "this angle"
t=11.7 AI highlights angle_17
t=14.1 student pointer enters angle_17
t=14.8 student asks "why is that 90 degrees?"
t=15.2 reference resolver selects angle_17
t=15.4 old plan cancelled
t=15.6 new explanation plan begins
```

Temporal history makes deictic language, interruption, debugging and lesson replay possible.

35. Lesson Replay and Deterministic Debugging

Every session should be replayable from the event log. This becomes a core engineering asset, not a reporting feature.

- Reproduce a failure exactly.
- Compare model versions on identical student interactions.
- Measure speech/canvas synchronization.
- Audit why a reference was resolved incorrectly.
- Generate training/evaluation data.
- Review student learning trajectories.
- Detect regressions after renderer or model changes.

A replay harness is essential if JAWVEE is expected to improve continuously without introducing silent regressions.

36. Evaluation Framework

Do not evaluate JAWVEE with a single “answer correctness” metric. Evaluate the entire grounded teaching loop.

Metric	Definition
Semantic render accuracy	Requested object semantics match actual scene
Geometric accuracy	Target coordinates/relations satisfy constraints
Reference accuracy	Student references resolve to intended entities
Temporal sync error	Difference between intended speech cue and visual cue
Math validity	Percentage of accepted transformations independently verified
Plan invalidation accuracy	Stale actions cancelled after interruption
Perceptual fidelity	Human/vision judgment of rendered output
Pedagogical correctness	Appropriate next teaching action
Abstention precision	Cases where system correctly refuses to guess
Recovery success	Successful correction after induced failures

37. Target Reliability Model

The phrase “100% accurate” should be translated into engineering guarantees.

37. Target Reliability Model

The phrase “100% accurate” should be translated into engineering guarantees.

Subsystem	Target
Object IDs / event integrity	100% deterministic
Renderer constraints	100% deterministic within supported primitives
Math validation	100% deterministic for supported operations
Scene-version consistency	100%
AI action schema validity	100% after validation
Pointer semantic targeting	100% for valid target IDs
Speech-to-cue association	Deterministic after audio alignment
Reference interpretation	Confidence-gated; never unconditional
Open-ended factual reasoning	Evidence/confidence-gated
External visual perception	Model-dependent; preserve uncertainty

38. Latency Budget

Teaching feels natural only if the system keeps the media path fast while deeper reasoning happens asynchronously where possible. Current realtime voice engineering explicitly separates media flow from slower reasoning/tool work to avoid blocking speech. ■cite■turn1search4■turn1search8■

Path	Design target
Student speech → live audio handling	Realtime / streaming
Reference resolution	Low-latency local/stateful query
Pointer hit testing	Local deterministic
Canvas mutation	Immediate deterministic command
Deep reasoning	Asynchronous when it does not block turn-taking
Verification	Immediate for local state; async for perceptual checks

39. Scalability Architecture

The architecture must scale along three axes: users, domains and model capability.

- **Users:** stateless media/API frontends where possible; session state in a durable event/state service.
- **Domains:** pluggable domain registries and validators.
- **Models:** model-independent tool contracts; replaceable realtime voice/reasoning/perception models.
- **Rendering:** deterministic client-side rendering from canonical scene operations.
- **Knowledge:** retrieval layer separated from lesson state.
- **Evaluation:** replayable event streams allow offline regression at scale.

40. Versioning and Concurrency

Every AI plan must declare the world version against which it was generated. Before execution, the Teaching Director checks whether the world has changed. If the version is stale, the plan is either safely rebased or discarded and regenerated.

```
Plan.base_version = 184

Current world = 187

If changed objects overlap plan targets:
invalidate plan
else:
```

```
rebase plan if action semantics remain valid
```

This prevents stale actions from affecting a changed canvas after student intervention.

41. Accessibility and Alternate Inputs

Semantic object identity enables accessibility. A screen reader, keyboard user or voice-only student can interact with the same object graph. The system should not depend on color alone for meaning; highlights and relationships need semantic labels.

Because visual actions are semantic, the same teaching plan can produce alternative modalities in the future: spoken description, tactile output, accessible textual diagrams or simplified views.

42. Security and Safety

- Validate all model tool calls against typed schemas.
- Do not allow arbitrary code execution through canvas actions.
- Enforce object ownership and session boundaries.
- Keep student raw audio/strokes subject to the product's privacy policy.
- Treat retrieved scientific claims as untrusted until source/evidence policy is satisfied.
- Do not allow a model to silently rewrite canonical lesson history.
- Maintain auditable provenance for generated explanations and external facts.

43. MVP → Production Roadmap

Phase	Scope	Exit criterion
0	Canonical scene graph + event log	Every canvas action is structured
1	Semantic renderer + AI tool API	AI can draw/point/highlight by object ID
2	Math semantic layer + validators	Math operations are independently verified
3	Teaching Performance Protocol	Speech and visuals share anchors/timeline
4	Teaching Director + interruption	Student can interrupt without stale actions
5	Reference resolver	this/that/these/spatial references grounded
6	Perception layer	Handwriting/freeform diagrams become hypotheses
7	Domain registry	Biology/science/chemistry capabilities plug in
8	Verification/replay	Every failure reproducible and repairable
9	Scale/evaluation	Automated benchmark + production telemetry

44. Corner-Case Matrix

Corner case	Required behavior
Child says “this” with two nearby objects	Use pointer + focus + recency; clarify if unresolved
Child says “that one” after zoom	Resolve in world coordinates, not screen pixels
Object moved after plan generation	Version conflict → rebase or regenerate
Student erases object AI is about to reference	Reference becomes invalid; cancel action
AI is speaking while student asks new question	Barge-in; preserve committed state; invalidate future queue
Student asks about an object created 30 seconds ago	Temporal retrieval + semantic graph
Same term appears 5 times	Use relation/position/attention; never rely on name alone
Student says “the blue line”	Filter by visual/style attributes + semantic compatibility
Student says “the line that crosses this”	Graph/topological relation + current referent

Corner case	Required behavior
Student's handwriting is ambiguous	Keep hypotheses; ask/confirm
Student writes an equation over existing text	Spatial overlap + z-order + stroke provenance
AI draws something incorrectly	Verifier catches state/constraint mismatch; rollback/repair
Math derivation is invalid	Symbolic validator rejects proposed step
Scientific explanation has multiple valid models	Retrieve evidence; expose assumptions/level
Question changes topic mid-animation	New intent wins; stale performance cancelled
Network disconnect during drawing	Client keeps canonical local event queue; reconcile on reconnect
Voice playback speed changes	Audio clock + semantic anchors preserve sync
TTS provider changes	Performance protocol remains provider-independent
New domain not known to core engine	Capability registry discovers domain plugin
Question has no useful visual representation	Explain verbally/textually; canvas may show abstract concept map
Question is unsafe/age-inappropriate	Safety/pedagogy policy can constrain plan before rendering

45. What “Nothing Hardcoded” Actually Means

It does not mean that the system contains no rules. A reliable educational system necessarily contains explicit schemas, mathematical invariants, safety policies, renderer constraints and domain knowledge. “Nothing hardcoded” should mean **no question-specific procedural patches**.

Bad hardcoding	Scalable replacement
if question contains 'sky blue' then draw rays	Concept graph + causal explanation planner
if user says '90 degrees' point to coordinates X/Y	Reference resolver + angle semantic object
if topic = DNA use fixed drawing script	Biology capability registry + semantic molecule graph
move cursor 500ms after phrase	Audio cue + semantic anchor
if renderer fails, redraw at x+5	Verification + geometry recomputation
if student asks 'why', use template #3	Intent + concept + pedagogy planner

46. Core Data Contracts

46.1 Semantic object

```
{
"id": "...",
"type": "...",
"domain": "...",
"properties": {},
"geometry": {},
"relations": [],
"provenance": {},
"confidence": 1.0,
"version": 17
}
```

46.2 Teaching plan

```
{
"base_world_version": 184,
"intent": "...",
"speech_segments": [],
"visual_actions": [],
"pointer_actions": [],
"camera_actions": [],
"verification": [],
"interrupt_policy": "replan"
}
```

46.3 Reference result

```
{
"expression": "that angle",
"candidates": [],
"selected": "angle_17",
"confidence": 0.97,
"evidence": [],
"policy": "execute"
}
```

47. Test Strategy

Testing must include both deterministic unit tests and adversarial multimodal scenarios.

- Render the same semantic scene at 1000 viewport sizes and verify geometric invariants.
- Generate thousands of paraphrases for the same reference intent.
- Test “this/that/these/two/one on the left” with distractor objects.
- Inject student interruptions at every word boundary of a teaching performance.
- Randomly delay network/tool responses and verify no stale actions execute.
- Mutate equations and verify symbolic validators reject invalid steps.
- Replay the same event log across renderer/model versions.
- Test ambiguity: require the system to abstain rather than select an incorrect target.
- Measure synchronization under different speech rates and network jitter.
- Test new domain capabilities without changing core orchestration code.

48. Evaluation Dataset Strategy

JAWVEE should build a proprietary evaluation corpus from replayable interactions, with explicit labels for reference targets, object semantics, correct visual actions, timing anchors, student intent and acceptable pedagogical responses.

Synthetic generation is useful for scale, but human-reviewed adversarial examples are required for ambiguous natural language. Research benchmarks demonstrate that current multimodal systems can perform substantially worse on genuine visual reasoning than on language-friendly proxies, so evaluation must measure the actual grounded task rather than only text answer quality. ■cite■turn0academia26■turn0academia24■

The dataset should deliberately contain distractors, ambiguous references, negative references (“not that one”), multi-hop spatial descriptions, topic changes, interruptions and objects with similar appearance.

49. Production Observability

Telemetry	Why
World version transitions	Detect stale-state bugs
Reference candidates/confidence	Diagnose grounding failures
Plan/action timestamps	Measure synchronization
Audio cue timestamps	Measure speech alignment
Renderer verification results	Detect visual failures
Plan cancellations	Understand interruption handling
Validator failures	Find reasoning/model issues
Abstention rate	Balance confidence vs usability
Repair rate	Measure system robustness
Human correction rate	Measure real-world quality

50. Product-Level Definition of Done

The feature is not production-ready merely because JAWVEE can draw equations. It is ready when the following loop is reliable:

```
Student asks
↓
System understands intent
↓
System resolves references against current world
↓
System reasons using appropriate domain capability
↓
System creates a teaching performance
↓
Speech + visuals are semantically linked
↓
Teaching Director executes
↓
Canvas renders deterministically
↓
Student may interrupt at any time
↓
System preserves actual state
↓
System resolves the new question
↓
Old plan is invalidated where necessary
↓
New performance is generated
↓
Every important action is verified
↓
World model remains consistent
```

51. Final Architectural Decision

JAWVEE should be built as a **semantic, event-sourced, spatial teaching environment** with a realtime voice interface—not as a whiteboard attached to an LLM.

The architecture should have five non-negotiable foundations:

- **Canonical Scene Graph:** exact objects, geometry, relationships and identities.
- **World + Lesson Model:** persistent semantic and temporal context.
- **Reference Resolver:** maps arbitrary natural language to scene entities using multimodal evidence and confidence.
- **Teaching Performance Protocol:** speech, drawing, pointer, camera and timing expressed as one plan.
- **Teaching Director + Verification:** executes, synchronizes, interrupts, invalidates, verifies and repairs.

The LLM supplies general reasoning and pedagogical decisions. It does not own geometry, timing, state integrity or deterministic computation.

This separation is the central mechanism for scaling from simple questions such as “what is a triangle?” to open-ended questions such as “why is the sky blue?”, “why is this angle 90 degrees?”, “why does this DNA base pair with that one?”, “what happens if I move this point?”, and eventually to unfamiliar questions the product team never anticipated.

The system generalizes because the unit of intelligence is not the question template. The unit is the **composition of concepts, relationships, capabilities, constraints and teaching actions**.

52. Research Findings and References

Tang, Mao & Suhr (EMNLP 2024), “Grounding Language in Multi-Perspective Referential Communication.” — Shows that multimodal referring-expression comprehension remains challenging and that grounding depends on shared scene/perspective context. ■cite■turn0search0■

Tumu, Shinde & Kordjamshidi (IJCNLP/AACL 2025), “Referring Expressions as a Lens into Spatial Language Grounding in Vision-Language Models.” — Highlights difficulties with ambiguity, complex spatial relations and negation in visual-language grounding. ■cite■turn0search1■

Deichler et al. (LREC 2026), “MM-Conv: A Multimodal Dataset and Benchmark for Context-Aware Grounding in 3D Dialogue.” — Uses synchronized speech, motion, gaze and scene geometry to evaluate dynamic reference grounding. ■cite■turn0search2■

Xu et al. (2025), “SpatialBench: Benchmarking Multimodal Large Language Models for Spatial Cognition.” — Reports a gap between basic perceptual grounding and higher-level symbolic reasoning, causal inference and planning. ■cite■turn0academia24■

VisuLogic (2025), “A Benchmark for Evaluating Visual Reasoning in Multi-modal Large Language Models.” — Reports substantial limitations in visual reasoning on carefully constructed problems. ■cite■turn0academia26■

Seitz, Lengfeld & Timofte (2025), “The Return of Structural Handwritten Mathematical Expression Recognition.” — Demonstrates the value of trace-to-symbol and structural representations for interactive handwritten math. ■cite■turn2academia3■

Olivastri et al. (2024), “Multi-Modal 3D Scene Graph Updater for Shared and Dynamic Environments.” — Supports dynamic multimodal scene graphs that update from human actions, perception and time. ■cite■turn2academia2■

OpenAI (2026), “How we built a realtime system for responsive voice AI in six months.” — Describes full-duplex media flow, asynchronous deeper reasoning, interruption handling and separation of media/application paths. ■cite■turn1search4■

OpenAI (2026), “Build more natural voice experiences with GPT-Live-1 in the API.” — Describes realtime voice, interruption handling, delegation and tool use. ■cite■turn1search1■

OpenAI (2026), “Advancing voice intelligence with new models in the API.” — Describes realtime audio models capable of reasoning, transcription and action during conversation. ■cite■turn1search3■

53. Final Recommendation to the JAWVEE Engineering Team

Do not start by trying to make the LLM “see the canvas better.” Start by making the canvas **know itself**.

Then make the lesson **know itself**. Then make speech, pointer and drawing refer to the same semantic entities. Then add perception only for information the structured system cannot know. Then put a Teaching Director between reasoning and execution. Finally, make every consequential action verifiable and every ambiguous interpretation abstainable.

If this order is followed, model improvements become additive rather than architectural rescues. A better LLM can improve explanation and planning. A better vision model can improve handwriting perception. A better TTS system can improve voice. A better renderer can improve visual fidelity. None of those improvements require rewriting the underlying educational state model.

That is the scalable path toward the JAWVEE experience you described: an AI teacher that can answer an essentially open-ended student question, decide what should be explained, construct the appropriate visual representation, speak while demonstrating it, point to exactly what it is discussing, respond to interruptions, and remain grounded in the same persistent world throughout the lesson.

One final engineering rule: never add a patch to fix a failure that should have been prevented by a missing abstraction. If a new class of question breaks the system, first ask which missing representation, capability, constraint, resolver, validator or orchestration primitive caused the failure. Add that general primitive—not a special case for the question.